

Hpc

```
#include <iostream>
```

```
#include <vector>
```

```
#include <omp.h>
```

```
using namespace std;
```

```
// Function to print array
```

```
void printArray(const vector<int>& arr) {
```

```
    for (int val : arr)
```

```
        cout << val << " ";
```

```
    cout << endl;
```

```
}
```

```
// Sequential Bubble Sort
```

```
void bubbleSortSequential(vector<int>& arr) {
```

```
    int n = arr.size();
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if (arr[j] > arr[j + 1]) {
```

```
                swap(arr[j], arr[j + 1]);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```

// Parallel Bubble Sort using OpenMP
void bubbleSortParallel(vector<int>& arr) {

    int n = arr.size();
    bool sorted = false;

    while (!sorted) {

        sorted = true;

        // Odd Phase
        #pragma omp parallel for shared(arr)
        for (int i = 1; i <= n - 2; i += 2) {

            if (arr[i] > arr[i + 1]) {

                swap(arr[i], arr[i + 1]);

                #pragma omp critical
                sorted = false;
            }
        }

        // Even Phase
        #pragma omp parallel for shared(arr)
        for (int i = 0; i <= n - 2; i += 2) {

            if (arr[i] > arr[i + 1]) {

                swap(arr[i], arr[i + 1]);
            }
        }
    }
}

```

```

        #pragma omp critical
        sorted = false;
    }
}
}
}

```

// Merge Function

```
void merge(vector<int>& arr, int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    vector<int> L(n1), R(n2);
```

```
    for (int i = 0; i < n1; i++)
```

```
        L[i] = arr[left + i];
```

```
    for (int i = 0; i < n2; i++)
```

```
        R[i] = arr[mid + 1 + i];
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    int k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k++] = L[i++];
```

```
        }
```

```
        else {
```

```
        arr[k++] = R[j++];  
    }  
}
```

```
while (i < n1) {  
    arr[k++] = L[i++];  
}
```

```
while (j < n2) {  
    arr[k++] = R[j++];  
}  
}
```

// Sequential Merge Sort

```
void mergeSortSequential(vector<int>& arr, int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;
```

```
        mergeSortSequential(arr, left, mid);
```

```
        mergeSortSequential(arr, mid + 1, right);
```

```
        merge(arr, left, mid, right);
```

```
    }  
}
```

// Parallel Merge Sort using OpenMP Tasks

```
void mergeSortParallel(vector<int>& arr, int left, int right, int depth = 0) {
```

```
    if (left < right) {
```

```

int mid = left + (right - left) / 2;

if (depth < 4) {

    #pragma omp task shared(arr)
    mergeSortParallel(arr, left, mid, depth + 1);

    #pragma omp task shared(arr)
    mergeSortParallel(arr, mid + 1, right, depth + 1);

    #pragma omp taskwait
}
else {

    mergeSortSequential(arr, left, mid);
    mergeSortSequential(arr, mid + 1, right);
}

merge(arr, left, mid, right);
}
}

int main() {

    int n;

    cout << "Enter the size of array: ";
    cin >> n;

    vector<int> arr(n);

```

```
cout << "Enter array elements:\n";

for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

vector<int> arr1 = arr;
vector<int> arr2 = arr;
vector<int> arr3 = arr;
vector<int> arr4 = arr;

// Sequential Bubble Sort
double start = omp_get_wtime();

bubbleSortSequential(arr1);

double end = omp_get_wtime();

cout << "\nSequential Bubble Sort Time: "
    << (end - start) << " seconds\n";

cout << "Sorted Array: ";
printArray(arr1);

// Parallel Bubble Sort
start = omp_get_wtime();

bubbleSortParallel(arr2);

end = omp_get_wtime();
```

```
cout << "\nParallel Bubble Sort Time: "  
    << (end - start) << " seconds\n";
```

```
cout << "Sorted Array: ";  
printArray(arr2);
```

```
// Sequential Merge Sort  
start = omp_get_wtime();
```

```
mergeSortSequential(arr3, 0, n - 1);
```

```
end = omp_get_wtime();
```

```
cout << "\nSequential Merge Sort Time: "  
    << (end - start) << " seconds\n";
```

```
cout << "Sorted Array: ";  
printArray(arr3);
```

```
// Parallel Merge Sort  
start = omp_get_wtime();
```

```
#pragma omp parallel  
{  
    #pragma omp single  
    mergeSortParallel(arr4, 0, n - 1);  
}
```

```
end = omp_get_wtime();
```

```

cout << "\nParallel Merge Sort Time: "
    << (end - start) << " seconds\n";

cout << "Sorted Array: ";
printArray(arr4);

return 0;
}

```

Output

Output-

```

[prlab@fedora ~]$ ./hpc_prno2
Enter the size of the array: 5
Enter 5 elements:
34 7 23 32 5
Sequential Bubble Sort Time: 1.344e-06 seconds
Sorted Array: 5 7 23 32 34
Parallel Bubble Sort Time: 0.000357863 seconds
Sorted Array: 5 7 23 32 34
Sequential Merge Sort Time: 4.747e-06 seconds
Sorted Array: 5 7 23 32 34
Parallel Merge Sort Time: 5.7565e-05 seconds
Sorted Array: 5 7 23 32 34
[prlab@fedora ~]$

```

Commands

Example Full Steps

Step 1: Open Terminal

Step 2: Go to File Location

Example:

```
cd Downloads
```

Step 3: Compile

```
g++ hpc.cpp -fopenmp -o hpc
```

Step 4: Run

```
./hpc
```

If g++ Not Installed

Run:

```
sudo dnf install gcc-c++
```